

AI AGENT 学习路线 · 学习专题 008

# 记忆与上下文工程 学习笔记（打印版）

---

11 个篇章

生成日期：2026-09-19 | 由 Markdown 笔记自动排版生成

# 目录

---

**01** 008 - 记忆与上下文工程 学习专题 [导航](#)

---

**02** 00 - 背景篇：Agent 记忆系统全景

---

**03** mem0 深度学习笔记

---

**04** 01 - 问题篇：为什么需要记忆层

---

**05** 02 - 概念篇：核心概念精讲（12 张概念卡片）

---

**06** 03 - 原理篇：两阶段流水线与存储架构

---

**07** 04 - 进阶篇：图记忆与论文精读

---

**08** 05 - 实战篇：从安装到本地化

---

**09** 06 - 生产篇：集成模式与最佳实践

---

**10** 07 - 对比篇：记忆方案横向评测与选型

---

**11** 08 - 自测篇：题库与面试题 [分级题库](#) ✓

---

共 11 个篇章。

# 008 - 记忆与上下文工程 学习专题

这是 AI Agent 学习路线中的记忆模块。记忆（Memory）与上下文工程（Context Engineering）是 Agent 与普通 Chatbot 的分水岭——**没有记忆的 Agent 每次醒来都是失忆状态。**

## 📖 本专题使用的学习方法（先读这段）

本专题笔记不是资料的堆砌，而是按**三种经过验证的学习方法**组织：

| 方法                            | 在笔记中的体现                                                | 你该怎么做                                      |
|-------------------------------|--------------------------------------------------------|--------------------------------------------|
| 费曼技巧 (Feynman Technique)      | 每个概念先用【一句话人话】+【生活类比】讲解，再进入技术细节                         | 读完后，尝试用自己的话把概念讲给同事听；讲不下去的地方就是你没懂的地方，回到笔记重读 |
| 黄金圈法则 (Why → What → How)      | 章节顺序：先讲"为什么需要" (01)，再讲"是什么" (02)，最后讲"怎么实现/怎么用" (03-06) | <b>不要跳过 Why 部分</b> ——知道为什么，细节才挂得住          |
| 康奈尔笔记法 + 主动回忆 (Active Recall) | 每章开头有【本节问题】清单（带着问题读），结尾有【本节自测】（合上笔记作答）                 | 读前扫一眼问题；读后 <b>合上笔记</b> 做自测，答错的标记下来         |
| 间隔重复 (Spaced Repetition)      | 08-自测篇 提供 D1/D3/D7/D21 复习计划                            | 按计划重做错题，而不是一次性刷完                           |

**给工程师的建议：**代码章节（05-08）不要只读，**一定要跑**。费曼技巧对代码同样适用：跑通后把示例删掉凭记忆重写一遍。

## 📖 专题目录

|                         |                                    |
|-------------------------|------------------------------------|
| 008-记忆与上下文工程/           |                                    |
| ├ README.md             | ← 你在这里：导航 + 方法论                    |
| ├ 00-背景篇-Agent记忆系统全景.md | ← 补充知识：认知科学→AI记忆分类→学术脉络            |
| ├ mem0/                 | ← 主案例：mem0 深度学习                    |
| │ ├ README.md           | 学习路线图 + 概念地图                       |
| │ ├ 01-问题篇-为什么需要记忆层.md  | Why：痛点场景 + token 经济学               |
| │ ├ 02-概念篇-核心概念精讲.md    | What：概念卡片（费曼式）                     |
| │ ├ 03-原理篇-两阶段流水线与存储.md | How：架构图解 + 伪代码                     |
| │ ├ 04-进阶篇-图记忆与论文精读.md  | 论文精读 + LOCOMO 数据 + 现役 Graph Memory |
| │ ├ 05-实战篇-从安装到本地化.md   | 全部代码 + 本地化部署 + 排错                  |
| │ ├ 06-生产篇-集成模式与最佳实践.md | 生产架构 + 框架集成 + 安全合规                 |
| │ ├ 07-对比篇-记忆方案横向评测.md  | 补充：Zep/Letta/LangMem 等 + 选型决策树     |
| │ └ 08-自测篇-题库与面试题.md    | 分级题库 + 间隔重复计划                      |

## 🎯 学习路径建议

| 目标           | 路径                                           | 预计投入   |
|--------------|----------------------------------------------|--------|
| 快速了解（能对话）    | README → 01 → 02 → 07 对比篇扫一眼                 | 1~2 小时 |
| 系统掌握（能选型+开发） | 全部顺序读 + 跑通 05 的代码 + 做完 08 的 L1/L2 题          | 1~2 天  |
| 深入精通（能架构设计）  | 精读 04 论文篇 → 06 生产篇落地实验 → 07 横向对比亲测 → 08 全部通过 | 1~2 周  |

## ∞ 与其他目录的关系

- 学习本专题前建议先过 002-llm基础（知道 token、上下文窗口是什么）
- 004-工具调用与function-calling 中的结构化输出思想，在 mem0 的 ADD/UPDATE/DELETE/NOOP 决策中直接用到
- 学完本专题，可以进 009-agent框架实战 看记忆如何接入 LangGraph / OpenAI Agents SDK

# 00 - 背景篇：Agent 记忆系统全景

【本节问题】① 人类的记忆是怎么分类的？② AI Agent 的记忆和人类记忆怎么对应？③ "记忆层"在整个 Agent 架构里处于什么位置？④ 这个领域有哪些代表工作，脉络是什么？

## 1. 为什么从认知科学讲起（费曼式铺垫）

给 Agent 设计记忆，本质是在**模仿人类大脑的记忆管理**。先花 3 分钟了解人类记忆，后面所有概念都有锚点。

**类比：把大脑想象成一家公司** - 前台（感觉记忆）：所有信息先经过这里，只停留几秒，99% 直接扔掉 - 工位便签（工作记忆/短期记忆）：正在处理的几件事，容量极小（心理学经典结论：约  $7 \pm 2$  个组块），关机就没了 - 档案室（长期记忆）：分三个柜子—— - **情景记忆** (episodic)：发生过的事。"上周三和张总开了会" - **语义记忆** (semantic)：事实与知识。"张总是客户方的技术负责人" - **程序记忆** (procedural)："怎么做"的技能，比如骑车、写周报的流程 - 遗忘曲线（艾宾浩斯）：不复习的记忆按指数衰减 → 启发：Agent 记忆也需要**衰减/淘汰机制**，否则垃圾越堆越多

## 2. 映射到 AI Agent：记忆分类学

| 人类记忆 | Agent 对应物              | 典型实现            | 生命周期   |
|------|------------------------|-----------------|--------|
| 工作记忆 | 上下文窗口内的对话历史、scratchpad | 直接放 prompt      | 单次会话   |
| 情景记忆 | 过往交互的摘要、事件日志           | 对话总结库           | 跨会话    |
| 语义记忆 | 用户画像、领域事实、知识           | 向量库/知识图谱        | 跨会话、长期 |
| 程序记忆 | Agent 学到的操作模式、技能流程     | 技能库、few-shot 模板 | 长期     |

另一条常用的切分维度：

Agent 记忆

- 短期记忆 (Short-term)
  - 对话历史 (conversation history)
  - 工作状态 (工具输出、中间变量)
  - 注意上下文 (当前任务的焦点)
- 长期记忆 (Long-term)
  - 事实型 (用户偏好、账户信息) ← mem0 的主战场
  - 情节型 (过去任务的总结)
  - 语义型 (概念关系、知识图谱)
  - 程序型 ("怎么做"的模式)

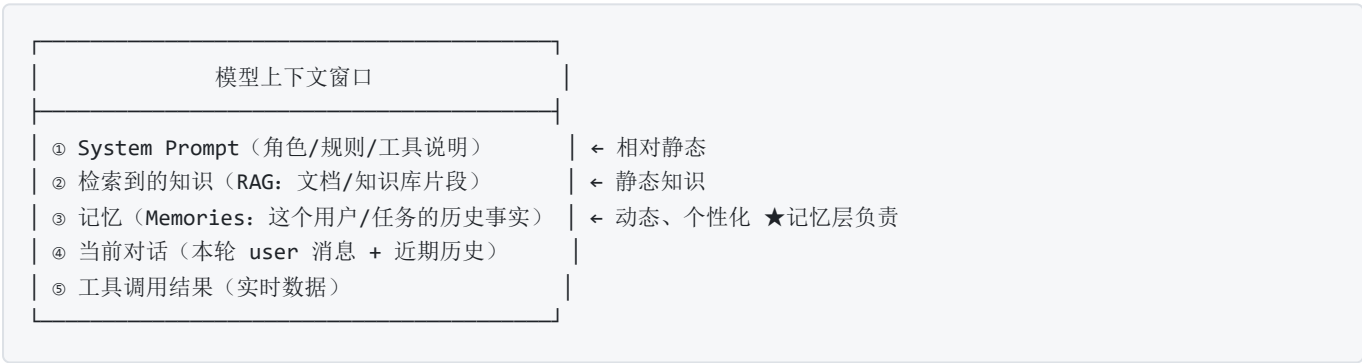
### 2.1 关键区分：会话记忆 vs 长期记忆

这是初学者最容易混的一对概念，用 harness (Agent 脚手架) 视角讲最清楚：

- **LLM 本身是无状态的**：每次调用独立，第 2 次调用看不到第 1 次的 prompt 和回复
- **会话记忆**其实是 harness 在维护：每轮结束后把 user 消息和回复**追加进 history**，下一轮**整段重发**给 LLM——状态在 harness 里，不在模型里
- 但这只在**同一段会话**内有效；新开会话，上一段内容不可达 → 跨会话失忆
- **长期记忆** = 挂在会话**外面**的独立存储：从每一段对话里沉淀事实、每一轮检索使用、跨会话持久。**务必与会话存储分开**（不同文件/不同 store）
- 以用户为中心的记忆可**跨 Agent 共享**：你的偏好可以同时被 ChatGPT、Claude 等不同 Agent 使用
- 生效方式两条路：给 Agent"搜记忆"的工具（explicit tool call），或每轮自动读写（automatic retrieval）

### 3. 记忆层在 Agent 架构中的位置（上下文工程视角）

**2024 年后业内形成共识：Agent 的核心工程问题是"上下文工程"（Context Engineering）**——在有限的上下文窗口里，决定"放什么、不放什么、以什么形式放"。一次 Agent 请求的上下文通常由五部分拼成：



**关键区分：RAG ≠ 记忆** - RAG 管**知识**：静态的、对所有人一样的语料（产品手册、政策文档） - 记忆层管**经历**：动态的、属于具体用户/会话的事实沉淀（"这个用户对花生过敏"） - 生产系统两者叠加使用（详见 mem0/06-生产篇）

### 4. 为什么需要专门的"记忆层"（三个反面方案）

| 反面方案     | 做法                | 死因                                       |
|----------|-------------------|------------------------------------------|
| A. 全量上下文 | 把所有历史塞进 prompt    | ① 超窗口；② 成本随历史线性涨；③ 延迟爆炸（见 01-问题篇的账本）     |
| B. 手工画像  | 自己设计 schema 存用户偏好 | 结构死板、只适合结构化事实，难以承载开放式对话沉淀；每个 Agent 重复造轮子 |
| C. 简单摘要  | 每轮让 LLM 总结历史      | 摘要丢细节、无法精确检索单条事实、越摘越长                    |

**记忆层的价值主张**：把"记忆管理"（提取→存储→检索→更新→遗忘）封装成独立基础设施，像数据库之于应用一样，让 Agent 框架只管推理。

## 5. 学术与工程脉络（读论文前的地图）

|         |                                                                                               |
|---------|-----------------------------------------------------------------------------------------------|
| 2023.10 | MemGPT (arXiv 2310.08560)<br>把 LLM 当操作系统，记忆分页管理 → 后发展为 Letta 平台                               |
| 2024    | LoCoMo 基准发布<br>长对话记忆评测数据集（10 段超长对话 × ~200 问）→ 后续所有记忆方案的比赛场                                    |
| 2024    | HippoRAG（神经生物学启发）/ A-Mem（自组织记忆）/ LangMem（LangChain 生态）<br>各种记忆机制百花齐放                          |
| 2025.04 | Mem0 论文 (arXiv 2504.19413) ★本专题主案例<br>两阶段管线（提取+更新），主打"生产就绪"：精度接近全量上下文，<br>延迟降 91%、token 省 90% |
| 2025+   | Zep Graphiti（时序知识图谱）、各家商业化平台成熟                                                                |
| 2026    | 竞争焦点转向：多模态记忆、记忆治理（合规/审计）、压缩引擎、评测标准                                                            |

**读文献的建议顺序：**先用 mem0（工程上手最友好）→ 再读 MemGPT 理解"操作系统式"思路 → 最后读 Zep/Graphiti 理解图谱路线。

## 6. 本专题的打开方式

本目录其余笔记以 **mem0** 为主线案例（工程上最普及、文档最全、开源可读），学完 mem0 的概念和架构后，再通过 07-对比篇 横向认识 Zep、Letta/MemGPT、LangMem 等，形成完整的选型判断力。

### 【本节自测】（合上笔记作答）

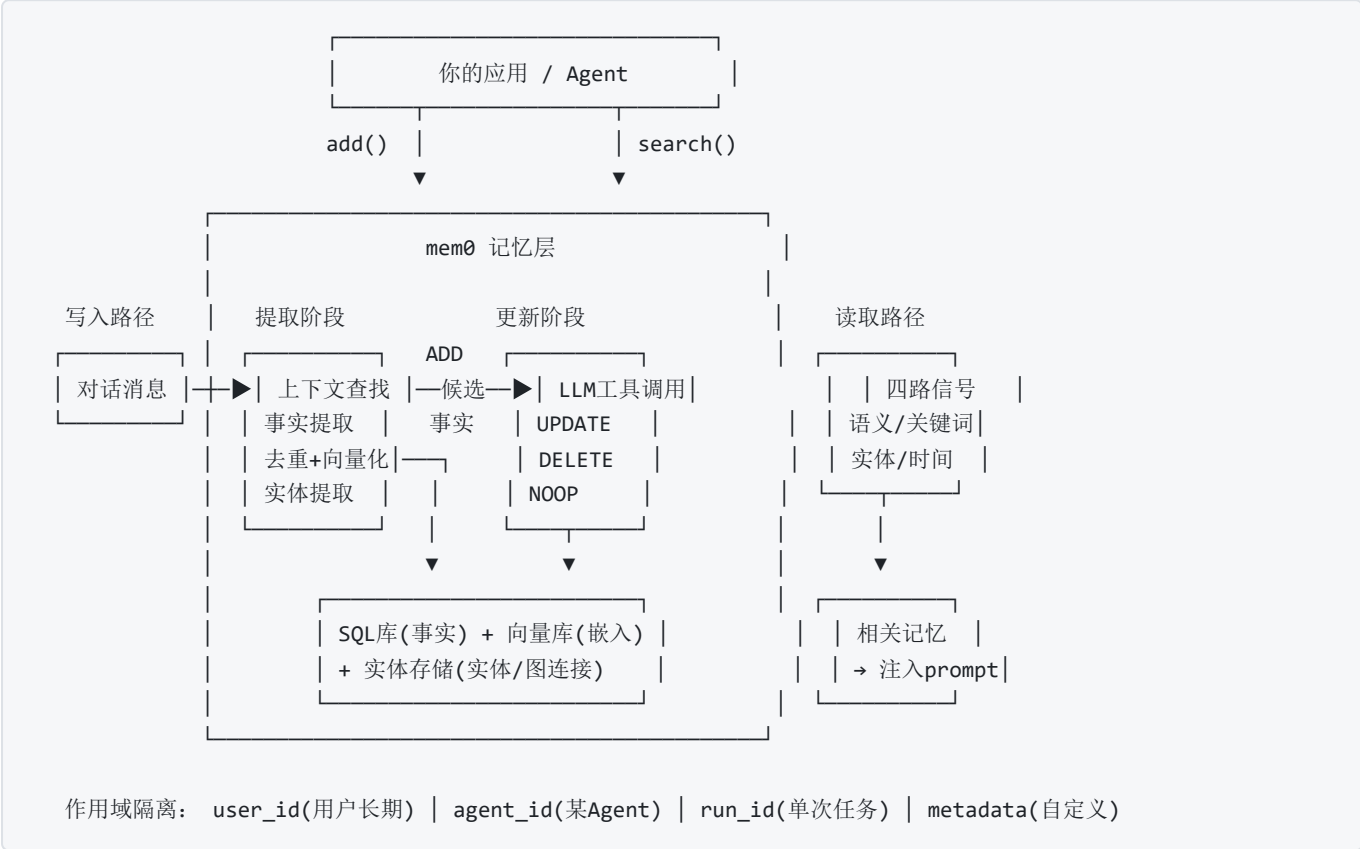
1. 人类长期记忆分哪三类？各自的 Agent 对应物是什么？
2. "上下文工程"一次请求的五块内容是什么？记忆层负责哪一块？
3. 用一句话说清 RAG 和记忆层的区别。
4. 手工画像方案为什么会被记忆层取代？
5. MemGPT 和 mem0 的核心思路差别是什么？（提示：操作系统分页 vs 两阶段事实提取）

答案都在本节内，答不出 3 题以上建议重读对应小节。



| 文件                     | 回答的问题    | 关键产出              |
|------------------------|----------|-------------------|
| <a href="#">08-自测篇</a> | 我真的学会了吗？ | L1~L4 题库 + 间隔复习计划 |

## 概念地图（全局鸟瞰）



## 间隔复习计划（给自己排期）

| 时点   | 动作                                      |
|------|-----------------------------------------|
| 今天   | 顺序读 01→03，跑通 05 的最小示例                   |
| D+1  | 只做 08 自测篇 L1 题；错题回炉                     |
| D+3  | 读 04 论文篇 + 07 对比篇；做 L2 题                |
| D+7  | 完成 06 的集成实验；做 L3/L4 题；给同事讲一遍 mem0（费曼检验） |
| D+21 | 重做全部错题；尝试在真实项目里落地一个记忆功能                 |

## 速查：最常用 API (详细见 05)

```
from mem0 import Memory
m = Memory()
m.add(messages, user_id="alex")           # 存
m.search("query", filters={"user_id": "alex"}) # 查
m.get_all(filters={"user_id": "alex"})      # 列
m.update(memory_id, "新内容")              # 改
m.delete(memory_id)                       # 删
m.history(memory_id)                      # 审计变更
```

# 01 - 问题篇：为什么需要记忆层

【本节问题】① 没有记忆的 Agent 在真实业务里会闹什么笑话？② "把历史全塞进上下文"的隐性成本到底有多大？③ 为什么手工存画像不行？

## 1. 黄金圈第一圈：Why——先看三个真实场景

### 场景 A：金鱼记忆的客服（重复盘问）

用户（第 1 天）：我对花生过敏，帮我推荐零食。  
客服 Agent：好的，为您推荐.....  
用户（第 7 天）：再推荐点零食。  
客服 Agent：好的！请问您有什么饮食禁忌吗？ ← 用户血压升高

病根：每轮对话都是孤立的，第 7 天的 Agent 对第 1 天一无所知。

### 场景 B：越聊越慢的长会话（成本与延迟失控）

一个长会话累积到 26k token（LoCoMo 基准的典型规模），如果每次请求都带上全部历史：

- 每次请求都为 26k token 付费，哪怕用户只问了一句"刚才说到哪了"
- 延迟直接起飞：全量上下文 p95 延迟 ~17 秒（mem0 实测数据）
- 上下文窗口终有上限，超出要么截断丢信息，要么直接报错

### 场景 C：摘要越摘越糊（信息损耗）

用"滚动摘要"省 token：每轮把历史压成摘要。结果——

- 摘要丢细节：用户第 2 轮说"预算 5 万以内"，第 30 轮摘要里只剩"客户有预算要求"
- 无法精确回忆：用户问"我上次说的过敏原是什么"，摘要答不上来
- 越摘越长、越摘越糊，还要为每次摘要调用付 LLM 费用

## 2. 算一笔账：全量上下文 vs 记忆层（数字说话）

以 LoCoMo 基准的长对话（~26k token）为例，mem0 论文实测：

| 方案        | 综合精度(J) | 总延迟 p95 | 每对话 token 开销       |
|-----------|---------|---------|--------------------|
| 全量上下文     | 72.9%   | 17.12s  | ~26k               |
| mem0 (向量) | 66.9%   | 1.44s   | ~7k                |
| mem0g (图) | 68.4%   | 2.59s   | ~14k               |
| Zep       | —       | —       | >600k (20 倍于原始对话！) |

**工程师视角的换算：**假设一个客服 Agent 每天处理 1 万轮长对话，按 \$3/百万 token 粗算：

- 全量上下文：1 万 × 26k × \$3/M ≈ **\$780/天**（这还只是记忆部分，且每轮都要重付）
- mem0：1 万 × 7k × \$3/M ≈ \$210/天，**省约 73%**；p95 延迟从 17s → 1.44s，用户体验量级差异
- 精度只从 72.9% → 66.9%（还高于几乎所有其他记忆方案），**用 6 个点的精度换 91% 延迟与 73% 成本**，绝大多数产品场景是笔划算的买卖

💡 费曼检验：能不能不看笔记，向同事解释“为什么记忆层比全量上下文便宜”？——因为事实只被**存一次**，之后每轮只注入**检索到的几条相关记忆**（几十上百 token），而不是**重付整个历史**的 token。

### 3. 为什么手工方案不行（反面教材逐个排雷）

| 手工方案                                           | 具体死法                                                                                        |
|------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>手工画像表</b> （user_profile 表：年龄/性别/偏好1/偏好2...） | ① 字段提前定死，开放式对话里冒出来的新信息没地方放；② 无法存“情节”（上周投诉过物流）；③ 每个 Agent 项目都要重写一遍这套管线                       |
| <b>把 DB 行直接塞 prompt</b>                        | 没有语义检索，无法回答“他是不是提过和预算有关的事”这类模糊问题                                                            |
| <b>每轮 LLM 总结</b>                               | 信息不可逆损耗 + 每轮额外 LLM 成本 + 无法按需取单条事实                                                           |
| <b>RAG 硬套</b> （把聊天记录当文档做向量检索）                  | 聊天记录里有大量寒暄、废话、时效性矛盾（“我不吃辣了”），原样入库后检索命中率极低、还可能检索出 <b>已被推翻的旧事实</b> ——记忆需要的是“提取+合并更新”，不是“原样索引” |

**结论：**记忆 ≠ 存储，记忆 = **提取（挑出值得记的）→ 整合（去重/更新/矛盾处理）→ 检索（按需召回）** 的完整生命周期管理。这正是 mem0 这样的记忆层存在的意义。

### 4. 记忆层的价值主张（一句话版）

把记忆管理的复杂性（提取、总结、检索、整合）抽象成一个简单 API，应用层只需 add() 和 search()，就能让 Agent 拥有跨会话、个性化、可控成本的记忆。

顺便划清边界（面试常考）：**mem0 不是 Agent 框架**——它不做编排、不做路由、不管工具执行，只做记忆。好处是可以插进任何 Agent 栈（LangChain、OpenAI Agents SDK、自研循环.....）。

### 5. 什么样的应用需要记忆层？（自查清单）

- [] 用户会多次回来使用（客服、助理、教育、健康、CRM）→ 需要 user 记忆
- [] 单次会话很长、多轮、多任务（深度咨询、调研）→ 需要会话/任务记忆
- [] 多个 Agent 共享或隔离上下文（多智能体系统）→ 需要 agent/org 记忆
- [] 上下文成本或延迟已经明显 → 需要记忆压缩与按需检索
- [] 以上都没有（一次性问答、纯工具执行）→ **不需要**，别为记而记

---

## 【本节自测】

---

1. 三种反面方案（全量上下文/手工画像/滚动摘要）各自的死因？
2. LoCoMo 长对话场景下，mem0 相比全量上下文省了多少延迟和 token？精度代价是多少？
3. 为什么"把聊天记录直接做 RAG"效果不好？
4. mem0 和 Agent 框架的边界是什么？这个边界带来什么好处？
5. 你的哪个项目适合加记忆层？用第 5 节清单过一遍。

## 02 - 概念篇：核心概念精讲（12 张概念卡片）

【本节问题】① 记忆四层分别是什么、什么时候用？② user\_id/agent\_id/run\_id 有什么区别？③ "消息"和"记忆"差在哪？④ 一条记忆对象长什么样？

每张卡片的读法：【一句话人话】→【类比】→【展开】→【易错点】。第一遍只看前两层，第二遍再看展开。

### 卡片 1：记忆层（Memory Layer）

- **一句话人话**：专门给 Agent 管"记什么、怎么记、怎么想起来"的基础设施。
- **类比**：Agent 是一个每天失忆的天才员工，记忆层是给他配的档案室 + 档案管理员。
- **展开**：独立于 Agent 框架的一层，对上提供 `add/search/update/delete` API，对下管理向量库、SQL、实体存储。应用只管调 API，不用操心存哪、怎么检索。
- **易错点**：记忆层 ≠ Agent 框架 ≠ RAG 框架。mem0 只管记忆，编排、工具、路由一概不管。

### 卡片 2：事实提取（Fact Extraction）

- **一句话人话**：从对话里挑出"值得长期记住的事"，写成干净的短句。
- **类比**：会议纪要员——不逐字记录，只把"决议、承诺、偏好"提炼成条目。
- **展开**：mem0 用 LLM 做提取。"我喜欢靠过道的座位" → 存成 `User prefers aisle seats`。默认开启（infer 模式）；也可关闭存原文。
- **易错点**：提取是**有损压缩**，所以重要原文要用 `infer=False` 保真。

### 卡片 3：四层记忆（Memory Types）

- **一句话人话**：便签（本轮）→ 日志（本次会话）→ 档案（这个用户）→ 图书馆（全公司）。
- **类比**：见上，范围逐层扩大、寿命逐层变长。
- **展开**：

| 层            | 生命周期  | 典型内容            | 怎么用                            |
|--------------|-------|-----------------|--------------------------------|
| Conversation | 单轮    | 工具输出、中间推理       | 不入长期库，轮次结束即弃                   |
| Session      | 分钟~小时 | onboarding 流程状态 | 挂 <code>run_id</code> ，结束自动过期  |
| User         | 数周~永久 | 偏好、账户信息         | 挂 <code>user_id</code> ，跨会话个性化 |
| Org          | 全局    | FAQ、产品目录、政策     | 多 Agent 共享，需维护者                |

- **易错点**：别把"本轮工具输出"这种一次性信息存进 user 层——记忆层最怕垃圾越积越多。

## 卡片 4：作用域三剑客 (user\_id / agent\_id / run\_id)

- **一句话人话：**给记忆贴的三种"归属标签"——记在谁名下、哪个 Agent 名下、哪次任务名下。
- **类比：**图书档案的"借阅人 / 部门 / 借阅批次"三联单。
- **展开：**
  - `user_id`：长期个性化，最常用
  - `agent_id`：同一个用户下，不同 Agent 各有记忆空间（又可通过共享 user 记忆互通）
  - `run_id`：**短期任务上下文，靠它实现"自动过期"**——文档原话：想自动过期就用 `run_id`
  - **易错点：**检索时**必须带作用域 filter**，否则不同用户/Agent 的记忆会串味（数据泄漏事故高发点！）。

## 卡片 5：消息 vs 记忆 (Messages vs Memories)

- **一句话人话：**你发进去的是聊天记录，存下来的不是聊天记录，而是提炼后的事实卡片。
- **类比：**上课听的是老师两小时讲话（消息），笔记本上留的是十行要点（记忆）。
- **展开：**

| 输入消息               | 存储的记忆                          |
|--------------------|--------------------------------|
| "我喜欢靠过道的座位"        | User prefers aisle seats       |
| "这个项目用 Postgres 吧" | Project decision: use Postgres |

- **易错点：**原样入库会污染检索（寒暄、口语、重复全在里面）。除非确需原文，保持 `infer` 开启。

## 卡片 6：infer 参数（提取开关）

- **一句话人话：**`infer=True`（默认）让 LLM 提炼后存；`infer=False` 一字不改存原文。
- **类比：**笔记员速记 vs 录音笔原样保留。
- **展开：**适合 `infer=False` 的场景——法律条款原文、配置片段、需要精确引用的内容。
- **易错点：**两种模式混用时注意：原文记忆不会被去重合并逻辑整理，会越积越多。

## 卡片 7：记忆对象 (Memory Object)

- **一句话人话：**每条记忆不是一行向量，而是一个有身份证、有分数、有出生日期的完整对象。
- **类比：**档案卡片 = 正文 + 标签 + 检索号 + 归档时间。
- **展开（真实返回结构）：**

```
{
  "id": "mem_123abc",           // 身份证: update/delete/history 都靠它
  "memory": "Name is Alex. Enjoys basketball and gaming.",
  "user_id": "alex",
  "categories": ["personal_info"], // 分类标签
  "created_at": "2025-10-22T04:40:22Z",
  "score": 0.89                 // 检索相关度分（四路信号融合的结果）
}
```

- **易错点：** `score` 是融合分，不是纯余弦相似度；不要自己再按 0.5 之类的阈值粗暴截断而忽略排序。

## 卡片 8: CRUD 与记忆生命周期

- **一句话人话：** 记忆不是只增不减的，而是像数据库记录一样可增、可改、可删、可查历史。
- **类比：** 档案卡片可以新立 (ADD)、补充 (UPDATE)、作废 (DELETE)，每次操作都有登记 (history)。
- **展开：** `add / search / get_all / get / update / delete / delete_all / history / reset` (详见 05-实战篇)。
- **易错点：** `update/delete` 需要先拿到 `memory_id`——通常从 `search` 或 `get_all` 的结果里取。

## 卡片 9: 自动提取是“增量式”的 (不会静默改写)

- **一句话人话：** 用户改主意了，`mem0` 会追加新事实，但不会偷偷把旧记忆改掉。
- **类比：** 档案管理员把新声明归档，但不动旧档案——改不改由你决定。
- **展开：** 用户说“我从奥斯汀搬到了西雅图”，新事实入库，旧事实仍在。需要修正 → 显式 `update/delete`；或依赖更新阶段的 DELETE 决策 (见 03-原理篇)。
- **易错点：** 以为 `mem0` 自动维护一致性而从不清理 → 检索可能同时召回新旧矛盾记忆。生产上要有治理策略。

## 卡片 10: metadata 与多租户 (Multi-tenant)

- **一句话人话：** 除了三剑客，你还能给记忆贴任意自定义标签，检索时按标签过滤。
- **类比：** 档案卡上的自定义角标：“华东区”、“售后业务”。
- **展开：** 典型用法——workspace、tenant、产品线、数据来源。生产上强烈建议显式规划，而非事后补。
- **易错点：** metadata 是精确过滤 (filter)，不是语义检索条件；“语义找相似 + filter 圈范围”要配合使用。

## 卡片 11: 异步摘要 (Asynchronous Summary)

- **一句话人话：** `mem0` 在后台持续维护一份“会话摘要”，供下一次提取时当上下文用。
- **类比：** 会议每半小时，助理更新一页“当前进展”贴在白板上。
- **展开：** 论文中的异步摘要生成模块——提取新事实时，携带最新摘要能判断“这条是不是新信息”。摘要异步进行，不阻塞主链路。
- **易错点：** 这是内部机制，你感知不到，但它是“提取质量”的关键保障——摘要过期会漏提/误判。

## 卡片 12: Reranker (重排序器)

- **一句话人话：** 向量检索粗排出一批候选后，请一个更精细的裁判把最相关的排到前面。
- **类比：** 初筛 (向量，快而糙) → 复审 (reranker，慢而准)。
- **展开：** OSS 默认不启用；可配 Cohere 等 provider。官方建议 `top_k` 10~20。
- **易错点：** 数据量小时收益不明显；先量检索命中率，再决定要不要加。

---

## 【本节自测】

---

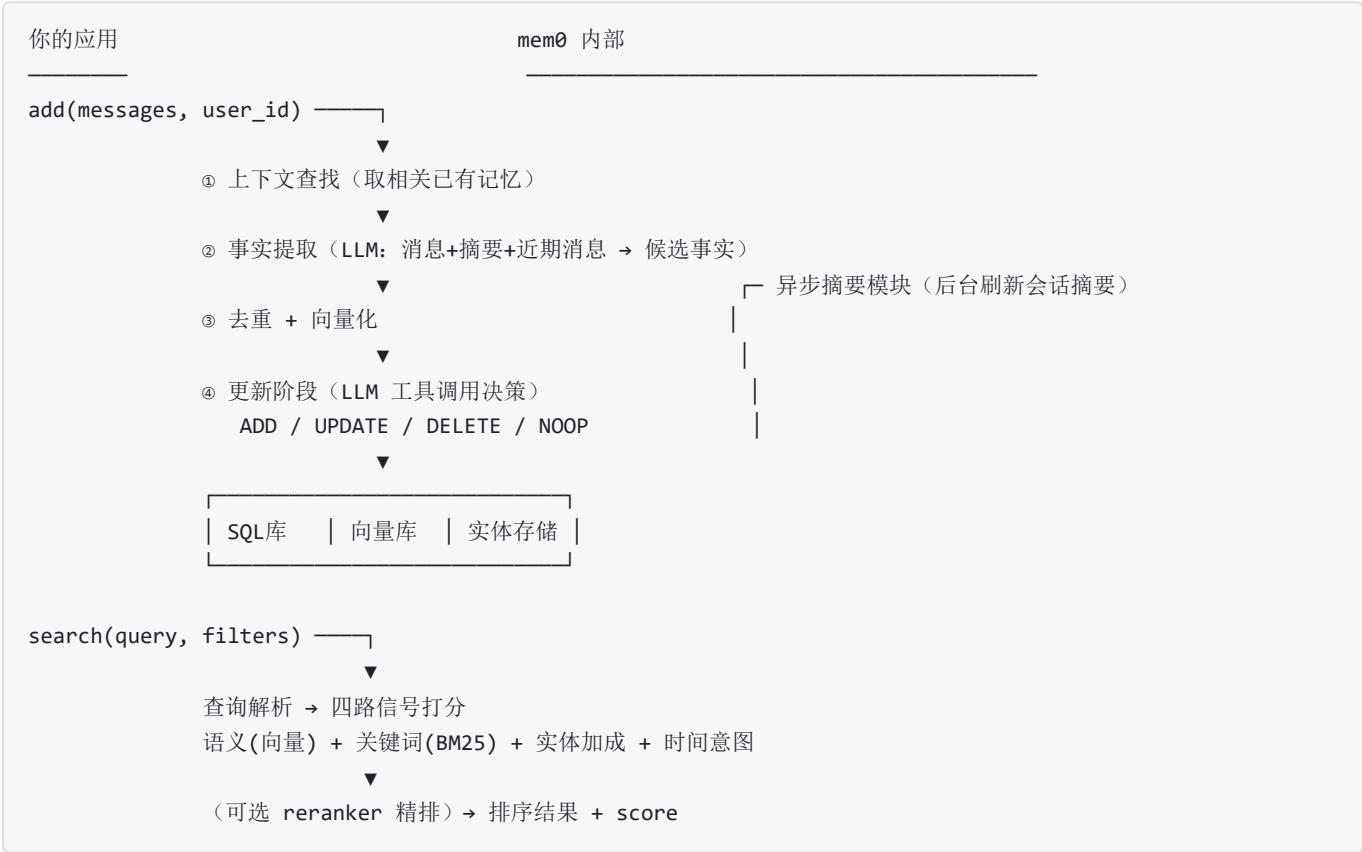
1. 四层记忆的划分和各自挂的作用域 ID?
  2. `infer=False` 什么时候用? 代价是什么?
  3. 为什么说"自动提取是增量式的"? 这带来什么生产责任?
  4. 一条记忆对象有哪六个关键字段? `score` 为什么不能当纯相似度用?
  5. 检索时忘带 `user_id` filter 会发生什么?
- 

**下一节预告:** 这些概念在 mem0 内部是怎么协同工作的? → 03-原理篇, 拆开看两阶段流水线。

# 03 - 原理篇：两阶段流水线与存储架构

【本节问题】① add() 一条消息后 mem0 内部发生了什么？② ADD/UPDATE/DELETE/NOOP 是怎么决定的？③ search() 的分数是怎么算出来的？④ 数据存在哪三件套里？

## 0. 全景图：一次对话的生命周期



**费曼式理解：**把 mem0 想象成一个档案管理员。1. 你递给他一页会议记录 (add) 2. 他先看看档案柜里已有有什么 (上下文查找)，免得重复立卡 3. 从记录里挑出要点写成卡片草稿 (事实提取) 4. 把草稿和相近的旧卡片摆在一起比对 (去重 + 更新决策)：- 全新的事 -> 立新卡 (ADD) - 补充旧卡的事 -> 在旧卡上添内容 (UPDATE) - 推翻旧卡的事 -> 作废旧卡 (DELETE) - 都是已知信息 -> 摇头放下 (NOOP) 5. 归档到三个柜子 (SQL 存卡片正文、向量库存"意思相近度"、实体存储记"这张卡提到谁")

## 1. 提取阶段 (Extraction Phase) 四步拆解

### 第 1 步：上下文查找 (Context Lookup)

- **做什么：**用新消息先查一遍已有记忆
- **为什么：**给后续步骤提供"已知信息"参照，避免重复入库
- **输入：**新消息 + 异步摘要模块维护的会话摘要 + 近期消息

## 第 2 步：事实提取 (Fact Extraction)

- **做什么**：LLM 从输入中抽取"可复用的持久事实"：偏好、决策、计划、账户信息、持久反馈
- **关键点**：LLM 被指示**忽略寒暄、临时性内容**，只留值得未来复用的
- **输出**：若干候选事实（短句），例如：

输入：「我下周要从奥斯汀搬到西雅图，新家想找个离公司近的健身房」

候选事实：

- User is moving from Austin to Seattle next week
- User wants a gym near the new home/office

**抽取 prompt 的上下文构成 (OSS 实现细节)** —— LLM 能抽出干净事实、并正确消解代词，靠的是 prompt 里拼了六样东西：

1. **角色**：memory extractor (记忆抽取器)
2. **用户摘要** (user summary)
3. **本次输入的 messages**：注意是整轮完整 messages，不是只抽 user 一句
4. **相关记忆**：把本批消息**展平成字符串** → embedding → 回主库搜相似记忆拼进上下文（帮 LLM 判断"哪些是新信息"）
5. **最近 10 条消息**（来自 SQLite）：专门用来**消解代词**——新消息是 "it is great" / "he's really good at this"，没有前几句就不知道 it / he 指谁，带上前几轮才敢写成"用户更喜欢素食餐厅"这种完整记忆
6. **对话日期 + 当前日期**（时间相关事实的判断依据）

## 第 3 步：去重 + 向量化 (Dedup & Embedding)

- **做什么**：合并冗余候选；为每条生成 embedding 存入向量库
- **为什么**：记忆库的价值 = 信噪比。垃圾记忆会让检索命中率崩塌

## 第 4 步：实体提取 (Entity Extraction)

- **做什么**：从每条记忆中抽出人名、地名、组织、产品、概念
- **为什么**：① 检索时实体重合可加成；② Platform 上实体直接构成 Graph Memory 的节点（见 04-进阶篇）
- **示例**：上例记忆的实体 = Austin、Seattle、gym

**⚠ 重要**：这条流水线是**增量式**的——"搬到西雅图"会作为新事实入库，旧的"住在奥斯汀"不会被静默改写。需要修正必须显式 update / delete，或依赖第 2 阶段的 DELETE 决策。

## 写入的三种模式 (add 的三种姿势)

| 模式                        | 行为                                 | 适用                             |
|---------------------------|------------------------------------|--------------------------------|
| procedural memory (程序性记忆) | 总结"这轮做了什么过程/用了哪些工具"，便于日后检索并复现同一套流程 | 现在不太常用——社区实践更倾向直接从对话里抽一条 skill |

| 模式                  | 行为                                        | 适用                 |
|---------------------|-------------------------------------------|--------------------|
| infer=false         | 输入消息直接 embed， <b>原样入库</b>                 | 表结构、法律条款等需要精确原文的内容 |
| infer=true（重点/默认姿势） | 触发完整 LLM 抽取管线，从 <b>整轮 messages</b> 中抽短句事实 | 长期 Agent 的标准做法     |

容易误解的点：一个 turn 里既有对话又有 tool call 时，不是“两种方式各存一次”——**从 user 到 final answer 的整段 messages（含 tool\_calls 和 tool 结果）作为一份输入**交给 infer=true；tool 过程只当上下文，不会再单独跑一遍 procedural。

## 2. 更新阶段（Update Phase）：四种操作怎么决定

对每条候选事实，mem0 从向量库取 **top-s 相似已有记忆**，让 LLM 通过**工具调用（function calling）**做出决策：

| 操作     | 判定条件     | 例子                     |
|--------|----------|------------------------|
| ADD    | 语义上全新    | 已知"喜欢篮球" → 新增"开始学钢琴"   |
| UPDATE | 对已有记忆的补充 | 已知"用户养了一只猫" → "猫叫小黑"   |
| DELETE | 与已有记忆矛盾  | 已知"用户是素食者" → "我开始吃牛排了" |
| NOOP   | 重复或无意义信息 | 再次自我介绍姓名               |

**易错点：**DELETE 依赖 LLM 判断矛盾，**不是 100% 可靠**（论文也承认这是局限）。金融、医疗等强一致场景，业务侧要有自己的冲突治理（如版本化 + 人工审核）。

## 3. 检索阶段（Retrieval Phase）：四路信号融合

search() 时，mem0 对候选记忆打**融合分**（Platform 上就是返回的那个 score）：

| 信号          | 机制                         | 擅长       | 例子                            |
|-------------|----------------------------|----------|-------------------------------|
| 语义 Semantic | 向量相似度                      | 概念性问题    | "他对吃的有什么讲究" ↔ "花生过敏"          |
| 关键词 Keyword | 词项匹配（BM25）                 | 精确名字/ID  | "订单 A1024 的状态"                |
| 实体 Entity   | query 中实体与记忆实体重合 → 加成      | 以人为中心的提问 | "Alice 负责什么" → 所有提到 Alice 的记忆 |
| 时间 Temporal | 写入时提取的时间元数据 vs query 的时间意图 | 时效性问题    | "他现在住哪"（倾向最新事实）               |

- **Platform：**四路全开，实体信号由内置 Graph Memory 驱动（自动、所有套餐可用）

- **OSS**: 向量库 + 可选 reranker; 实体信号仅做重合加成, **无图记忆**
- **易错点**: `score` 是融合分, 直接拿来做"绝对阈值"判断不可靠, 按相对排序取 top-k 更稳。

## OSS 检索管线细节 (社区源码解析, 具体数值随版本可能变化)

**两种触发方式** (后面管线相同, 差别只在谁发起 Memory search) :

- **Explicit tool call**: Agent 自己决定调用"搜索记忆"工具
- **Automatic retrieval**: 用户消息一发, harness 自动拿消息当 query 检索

**打分流水线** (一个最容易被误解的点: 关键词和实体**不能往候选池里加新记忆**, 只有 ANN 语义检索负责"出池") :

1. **embed query** (必须用与建库**同一个 embedding 模型**) → ANN 召回语义相近记忆; 实际召回条数 =  $\max(\text{topK} \times 4, 60)$  (topK=10 时实际取 60 条候选)
2. **池内并行打分** (只在这 60 条上做) : - **关键词分**: query 与记忆都做词形还原 (lemma 在写入时就已存), 比词重叠 (BM25), 映射到 0~1 - **实体加分**: 从 query 抽实体 → 实体库命中 → 只保留**已在池内**的记忆, 加 0~0.5 分; **实体挂的记忆越少, 分越高**——挂 1000 条的 "Paris" 不如只挂 2 条的小众实体相关
3. **三路融合**: 语义(0~1) + 关键词(0~1) + 实体(0~0.5) = 满分 **2.5**, 相加后 ÷ **2.5** 归一, 按最终分取真正的 topK 返回

**query 改写**: mem0 默认**不做** query rewriting; 想提升检索, 在 **harness** 侧加一层——用户提问先用小 LLM 改写成更好的 query, 再送进检索管线。

**定位**: 这本质上是一套 RAG (召回→打分→截断), 但**不是**"语义检索 || 关键词检索各自召回 → cross-rank"的形态——BM25 和实体加分只在 ANN 池内打分。

## 4. 存储三件套 (Where memories live)

| 存储             | 存什么             | 支撑什么查询              | OSS 对应                                        |
|----------------|-----------------|---------------------|-----------------------------------------------|
| <b>SQL 数据库</b> | 事实正文 + metadata | 精确取单条、审计            | 默认 SQLite ( <code>~/.mem0/history.db</code> ) |
| <b>向量数据库</b>   | embeddings      | 语义相似                | 默认本地 Qdrant ( <code>/tmp/qdrant</code> )      |
| <b>实体存储</b>    | 记忆中抽出的实体        | 实体加成; Platform 上支撑图 | OSS 有实体加成, 无图                                 |

**架构特点 (面试加分点)** : mem0 **运行时无状态** (服务可水平扩展), **存储层持久** (每个 user\_id/agent\_id 的记忆跨会话演进); 记忆衰减、摘要等重活**异步**进行, 保证检索亚秒级响应。

**OSS 实现细节 vs 官方文档口径**: 在 OSS 里, 记忆正文 + 元数据实际存在**主向量库**中, 元数据字段相当丰富——创建/更新日期、**归属** (用户侧记忆如"更喜欢素食餐厅", 还是 Agent 侧记忆如学到的时事/流程)、**hash** (只用于**措辞完全相同**的去重, 不是最精细的方案)、**词形还原 (lemma)** 版本 (供关键词检索)、**过期日期**、**创建者 attribution** (多 Agent 场景记录是哪一个是写的); 实体抽出到**实体向量库**并链回主记忆 (问"巴黎"→先命中实体 Paris →带回链上的全部主记忆)。而 **SQLite** 承担两件并列的事:

① 增删改**变更日志**（对账、回放、溯源"这条记忆怎么来的"）；② 保存**最近 10 条消息**给抽取器消解代词。两种口径不矛盾：Platform 把事实+元数据的 source of truth 收敛到 SQL 层；OSS 是"向量库为主、SQLite 记账"。

## 5. 伪代码：把两阶段写出来（理解用，非真实源码）

```
# ===== add() 内部（简化伪代码） =====
def add(messages, user_id, ...):
    summary = get_or_refresh_summary(user_id)          # 异步摘要模块
    related = vector_search(messages, user_id)         # ① 上下文查找
    candidates = llm_extract(messages, summary, related) # ② 事实提取
    candidates = dedup(candidates)                     # ③ 去重
    for fact in candidates:
        emb = embed(fact)
        similar = vector_topk(emb, s)                  # 取 top-s 相似记忆
        op = llm_decide(fact, similar)                 # ④ 更新决策（工具调用）
        if op == ADD: insert_sql(fact); vector_upsert(emb)
        if op == UPDATE: merge_and_rewrite(similar[0], fact)
        if op == DELETE: mark_deleted(similar[0])
        if op == NOOP: pass
    entities = llm_extract_entities(candidates)         # 实体提取 → 实体存储
    async_refresh_summary(user_id)                     # 摘要后台刷新

# ===== search() 内部（简化伪代码） =====
def search(query, filters, limit=10):
    q_emb = embed(query)
    cand_s = vector_search(q_emb, filters)              # 语义召回
    scores = fuse(
        semantic = cosine(q_emb, cand_s),
        keyword  = bm25(query, cand_s),
        entity   = entity_overlap(query, cand_s),      # Platform: 图加成
        temporal = time_intent(query, cand_s),
    )
    return rerank_topk(cand_s, scores, limit)           # 可选精排
```

### 【本节自测】

1. 提取阶段四步的顺序和各自目的？第 1 步为什么要带上"会话摘要"？
2. UPDATE 和 ADD 的判定边界？举一个会触发 DELETE 的对话例子。
3. 四路检索信号分别解决什么问题？OSS 和 Platform 在这里差在哪？
4. 存储三件套各存什么、支撑什么查询？
5. 为什么说 mem0 "运行时无状态、存储层持久"？这对部署意味着什么？
6. （费曼检验）向同事复述档案管理员类比，覆盖 ADD/UPDATE/DELETE/NOOP。

**下一节预告：**图记忆在论文里长什么样？和现役产品有什么区别？→ 04-进阶篇。

# 04 - 进阶篇：图记忆与论文精读

【本节问题】① 论文里 Mem0 和 Mem0g 的架构差异？② LOCOMO 实验说明了什么、没说明什么？③ 现役产品的 Graph Memory 和论文里的 Mem0g 是一回事吗？④ 旧代码迁移要注意什么？

## 1. 论文基本信息

《Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory》 - arXiv: 2504.19413 (2025-04) - 作者: Prateek Chhikara, Dev Khant, Saket Aryan 等 (mem0 团队) - 两个贡献: ① Mem0 主架构 (两阶段管线 + 向量库) ; ② Mem0g 图记忆扩展; ③ LoCoMo 系统评测

## 2. Mem0 主架构 (论文视角, 与 03-原理篇对应)

- **提取阶段**: 当前消息对 + 会话摘要 + 近期消息 → 产出候选记忆 (显著事实)
- **异步摘要模块**: 并行持续刷新会话摘要, 保证提取时全局上下文最新
- **更新阶段**: 候选事实 + top-s 相似记忆 → LLM 通过**工具调用**决定 ADD/UPDATE/DELETE/NOOP
- **类比锚点**: 人类 = 便签本 (近期对话) + 归档柜 (蒸馏后的事实), 存的是"要点"不是"录音"

## 3. Mem0g 图记忆架构 (论文版)

- **表示**: 有向标记图  $G=(V, E, L)$ : 节点=实体, 边=关系三元组 (含类型标签, 如 用户A -[偏好]-> 早上会议)
- **构建**: 两阶段 LLM 管线——**实体抽取器** (识别人/地/物/概念/事件 + 重要性分析) → **关系生成器** (产出关系三元组), 再经冲突检测后更新 **Neo4j** 图
- **检索 (双路)**: 1. **实体中心法**: 定位关键实体 → 沿边探索子图 → 拼装相关上下文 2. **语义三元组法**: query 与图三元组做 embedding 匹配
- **强项**: 结构化保留关系与时序 → **时序推理**最好的记忆方案之一

## 4. LOCOMO 实验 (重点数据 + 正确解读)

**实验设置**: LoCoMo 数据集 = 10 段超长对话 (平均 600 轮、~26k token) × 每段 ~200 问题; 四类题型: 单跳 / 多跳 / 时序 / 开放域; 指标: J (LLM-as-a-Judge)、F1、BLEU-1 + 部署指标 (延迟、token)。

### 4.1 总表

| 方案         | 综合 J   | 特点                                |
|------------|--------|-----------------------------------|
| 全量上下文 (上界) | 72.90% | p95 延迟 17.12s、~26k token/轮, 成本不可行 |
| Mem0g (图)  | 68.44% | 时序/开放域强                           |

| 方案        | 综合 J          | 特点                   |
|-----------|---------------|----------------------|
| Mem0（向量）  | <b>66.88%</b> | 延迟/token 最低          |
| 最强 RAG 配置 | ~61%          | —                    |
| A-Mem     | ~52.9%        | Mem0 比 A-Mem 高 13.98 |
| LangMem   | 58.10%        | —                    |
| OpenAI 记忆 | 52.90%        | Mem0 单跳比它高 3.34      |

4.2 分项亮点

| 题型  | 冠军           | 数据                       |
|-----|--------------|--------------------------|
| 单跳  | Mem0         | J=67.13（+3.34 vs OpenAI） |
| 多跳  | Mem0         | J=51.15                  |
| 时序  | <b>Mem0g</b> | F1=51.55 / J=58.13       |
| 开放域 | Zep          | J=76.60（Mem0g 75.71 紧随）  |

4.3 效率数据（工程最关心）

- 检索延迟：Mem0 p50 **0.148s** / p95 **0.200s**
- 总延迟：Mem0 中位 **0.708s**；p95 **1.44s**（比全量上下文低 **91%**）
- 记忆 token 开销/对话：Mem0 ≈ **7k**，Mem0g ≈ **14k**（图表示翻倍），Zep >**600k**（比原始对话还大 20 倍）
- 图构建时效：Mem0g 一分钟内完成 vs Zep 数小时延迟

4.4 正确解读（防误导）

- ✓ "精度接近全量上下文、成本延迟低一个量级"——成立，这是核心结论
- ⚠ 精度**没有**超过全量上下文（66.9/68.4 vs 72.9），别被营销话术带偏
- ⚠ 基准是**对话记忆**场景，不等于你的业务场景；官方建议用自有数据评测（见 06-生产篇）
- ⚠ 论文自认局限：开放域略逊 Zep；仅文本；冲突检测全靠 LLM；未融合分层记忆

5. 现役 Graph Memory vs 论文 Mem0g（关键辨析！）

这是两代东西。读旧教程最容易在这里翻车：

| 维度  | 论文 Mem0g（2025-04）  | 现役 Platform Graph Memory（2026） |
|-----|--------------------|--------------------------------|
| 存储  | 外接 Neo4j 图数据库      | 平台内置，无需部署任何图库                  |
| 边类型 | 带类型三元组（LLM 生成关系标签） | 无 schema、无类型标签：共享实体的记忆互连（共现推断） |

| 维度   | 论文 Mem0g (2025-04)                                      | 现役 Platform Graph Memory (2026)                                 |
|------|---------------------------------------------------------|-----------------------------------------------------------------|
| 构建方式 | 实体抽取器 + 关系生成器两阶段管线                                      | 实体提取 → 实体成为节点 → 共现实体的记忆建立连接                                     |
| 检索方式 | 实体中心子图探索 + 三元组语义匹配                                      | 实体匹配 → 命中记忆 <b>加成</b> ，融合进 <code>score</code> （与向量+BM25 一并计算）   |
| 返回形式 | <code>relations</code> 字段（关系列表）                         | 无单独图负载，只影响排序； <code>relations</code> 恒为空列表                      |
| 开关   | <code>enable_graph</code> + <code>graph_store</code> 配置 | <b>始终开启</b> ，无需配置； <code>enable_graph</code> 已废弃（仅为兼容保留，不恢复旧行为） |
| 可用范围 | 研究实现                                                    | Platform 全套餐； <b>OSS 无</b> ；交互式 Graph 可视化仅 Pro/Enterprise       |

**现役 Graph Memory 什么时候有用：** - 实体中心问题："我们对 Alice 都知道些什么？" → 把散落在多条记忆里的 Alice 事实连起来 - 多跳召回：一条记忆里的事实连到另一条记忆里的事实 - 成本：零——不用定义 schema、不用标注关系

**旧代码迁移清单**（从外接图库时代升级）： - [] 删除 `graph_store` 配置块与 `enable_graph=True` - [] 业务代码若读取返回中的 `relations` 字段 → 必须改造（现在恒为 []） - [] 依赖"带类型关系"做推理的场景 → 现役共现图**不提供**关系语义，需自行评估 - [] OSS 用户需要图能力 → 考虑 Zep Graphiti（开源）或上 Platform

## 6. 论文精读路线（给要精读原文的你）

| 论文章节         | 重点             | 对应本笔记   |
|--------------|----------------|---------|
| §3.1 Mem0 架构 | 两阶段管线图、工具调用机制  | 03-原理篇  |
| §3.2 Mem0g   | 图构建与双路检索       | 本篇 §3   |
| §4 实验        | LoCoMo 设置、基线选择 | 本篇 §4   |
| §4.x 效率      | 延迟/token 对比图   | 本篇 §4.3 |
| §5 讨论        | 局限与未来方向        | 本篇 §4.4 |

### 【本节自测】

- Mem0g 的检索双路是什么？为什么它时序推理强？
- 全量上下文在 LoCoMo 上精度最高，为什么生产上不推荐？给出至少三个理由。
- 现役 Graph Memory 的边是怎么来的？和论文 Mem0g 的边有什么本质区别？
- 你维护的老代码在用 `relations` 字段，升级后会发生什么？怎么办？
- 哪类问题应该选 Mem0g/图记忆，哪类问题选基础 Mem0 就够？

---

**下一节预告：**原理都懂了，动手把 mem0 跑起来 → 05-实战篇。

## 05 - 实战篇：从安装到本地化

【本节问题】① 怎么 5 分钟跑通？② 全部 API 怎么用？③ 怎么换成 Ollama + Chroma 本地方案？④ 报错了怎么查？

### 1. 环境准备

```
# Python 3.10+, 安装 SDK
pip install mem0ai

# 用 Qdrant 时需要客户端 + 启动服务（Chroma 则零部署）
pip install qdrant-client
docker run -p 6333:6333 qdrant/qdrant

# 默认走 OpenAI, 先配 Key
export OPENAI_API_KEY="sk-..."
```

### 2. 最小示例（默认配置，5 分钟）

```
from mem0 import Memory

m = Memory()

# 1) 存：传入对话消息 + 用户标识
messages = [
    {"role": "user", "content": "Hi, I'm Alex. I love basketball and gaming."},
    {"role": "assistant", "content": "Hey Alex! I'll remember your interests."},
]
m.add(messages, user_id="alex")

# 2) 查：带作用域过滤
results = m.search("What do you know about me?", filters={"user_id": "alex"})
print(results)
```

返回结构（对照 02-概念篇卡片 7）：

```
{
  "results": [
    {
      "id": "mem_123abc",
      "memory": "Name is Alex. Enjoys basketball and gaming.",
      "user_id": "alex",
      "categories": ["personal_info"],
      "created_at": "2025-10-22T04:40:22.864647-07:00",
      "score": 0.89
    }
  ]
}
```

**默认装配清单**（排错时心里要有这张表）：

| 组件         | 默认值                                                 |
|------------|-----------------------------------------------------|
| LLM        | OpenAI <code>gpt-5-mini</code> （提取 + 更新决策）          |
| Embeddings | OpenAI <code>text-embedding-3-small</code> （1536 维） |
| 向量库        | 本地 Qdrant, 数据落盘 <code>/tmp/qdrant</code>            |
| 历史库        | SQLite <code>~/.mem0/history.db</code>              |
| Reranker   | 未启用                                                 |

### 3. Platform 版（托管，零基础设施）

```
import os
from mem0 import MemoryClient

client = MemoryClient(api_key=os.getenv("MEM0_API_KEY")) # app.mem0.ai 获取

client.add(messages, user_id="user123")
results = client.search(
    "What are my dietary restrictions?",
    filters={"user_id": "alice"},
)
```

区别：向量库/reranker 全托管、Graph Memory 自动开启、有审计日志与 workspace 治理；高级功能（webhooks、多模态、自定义分类）即开即用。

## 4. CRUD 全 API 实操

```
from mem0 import Memory
m = Memory()

# ---- 存 ----
m.add(
    [{"role": "user", "content": "我在做 CTRM 系统，偏好用 Vue 2.7"}],
    user_id="dev001",
    metadata={"team": "trading"},          # 自定义过滤标签
    infer=True,                          # 默认: LLM 提取事实
)
m.add(
    [{"role": "user", "content": "合同额度表结构: info_customer_contract_limit..."}],
    user_id="dev001",
    infer=False,                          # 存原文，适合表结构/条款类内容
)

# ---- 查 ----
m.search("他偏好什么前端框架?", filters={"user_id": "dev001"}) # 语义检索
m.get_all(filters={"user_id": "dev001"})                        # 列出全部

# ---- 改 / 删 ----
r = m.search("Vue", filters={"user_id": "dev001"})
mid = r["results"][0]["id"]
m.update(mid, "偏好 Vue 2.7, 正计划迁移 Vue 3")                # 显式修正
m.delete(mid)                                                    # 删单条
# m.delete_all(filters={"user_id": "dev001"})                  # 清空某作用域

# ---- 审计 ----
m.history(memory_id=mid)    # 看这条记忆的 ADD/UPDATE/DELETE 全过程
```

## 5. 自定义配置（换模型 / 换向量库 / 加 reranker）

```
from mem0 import Memory

config = {
    "vector_store": {
        "provider": "qdrant",
        "config": {"host": "localhost", "port": 6333},
    },
    "llm": {
        "provider": "openai",
        "config": {"model": "gpt-5-mini", "temperature": 0.1},    # 提取温度 ≤ 0.2
    },
    "embedder": {
        "provider": "openai",
        "config": {"model": "text-embedding-3-small"},
    },
    "reranker": {
        "provider": "cohere",
        "config": {"model": "rerank-v3.5"},    # 需 COHERE_API_KEY; top_k 建议 10~20
    },
}

memory = Memory.from_config(config)
```

也支持 YAML: `config = yaml.safe_load(open("config.yaml"))` 后传入 `from_config`。

### Provider 速查表 (Python)

| 组件       | 可选 provider                                                                                        |
|----------|----------------------------------------------------------------------------------------------------|
| LLM      | openai, anthropic, gemini, groq, <b>ollama</b> , aws_bedrock, azure_openai, <b>litellm</b>         |
| Embedder | openai, gemini, azure_openai, <b>ollama</b> , huggingface, vertexai, aws_bedrock                   |
| 向量库      | <b>qdrant</b> , <b>pgvector</b> , <b>chroma</b> , pinecone, redis, weaviate, milvus, elasticsearch |

💡 国内项目常见组合: LLM 用 `litellm` (转发 DeepSeek/Qwen/GLM) 或 `ollama` (本地), 向量库用 `chroma` (零部署) 或 `pgvector` (复用现有 PostgreSQL)。

## 6. 本地化方案 (Ollama + Chroma, 断网可跑)

```
ollama pull llama3.1          # 或 qwen2.5 等中文友好模型
pip install chromadb
```

```
config = {
    "llm": {
        "provider": "ollama",
        "config": {
            "model": "llama3.1",
            "temperature": 0.1,
            "ollama_base_url": "http://localhost:11434",
        },
    },
    "embedder": {
        "provider": "ollama",
        "config": {"model": "nomic-embed-text"},
    },
    "vector_store": {
        "provider": "chroma",
        "config": {"collection_name": "agent_memory"},
    },
}
memory = Memory.from_config(config)
```

**本地化的代价：**小模型的提取/更新决策质量明显低于 GPT 级模型，事实漏提、误判增多——生产要权衡，或"提取用强模型、检索用本地"混合。

**本地模型怎么选（社区实践建议）**

- **抽取 LLM：**抽取是结构化任务，**1B~12B 小模型即可胜任**（别低于 1B，除非微调过）；Hugging Face 按 Text Generation 筛选。推荐 **Qwen3 8B** 一档（中文友好；mem0 云端常见搭配是 GPT-5 Mini 量级）
- **Embedding**（Feature Extraction 类）候选（四条互为备选，不是流水线）：

| 模型                             | 特点                          |
|--------------------------------|-----------------------------|
| BAAI/bge-m3                    | 多语 · dense + sparse（中文项目首选） |
| intfloat/e5-large-v2           | 英语检索强                       |
| nomic-ai/nomic-embed-text-v1.5 | 长上下文 embedding              |
| gte-Qwen2-1.5B-instruct        | instruction-aware 检索        |

- 选型看**对应领域/语言的 embedding 榜单**（多语、英语、医疗等各有榜单）
- **长期路线：**微调自己的小模型，本地效果更稳

来源补充：[Agent 长期记忆 \(以 Mem0 为例\)](#)——含源码级细节与本地选型的社区文章，具体数值随 SDK 版本可能变化。

## 7. Node.js 版要点

```
import { Memory } from "mem0ai/oss"; // ⚠ OSS 必须从这个入口导入

const memory = new Memory({
  version: "v1.1",
  embedder: { provider: "openai", config: { model: "text-embedding-3-small" } },
  // TypeScript 配置键为 camelCase
});
```

- provider SDK 是**可选 peer 依赖**：用 Qdrant 要 `npm install @qdrant/js-client-rest`，只装 `mem0ai` 不带它们
- 报 `Cannot find module` 九成是入口写错或缺 provider 依赖

## 8. 排错速查表

| 现象                            | 原因                 | 解决                                        |
|-------------------------------|--------------------|-------------------------------------------|
| Qdrant 连接失败                   | 6333 未暴露 / key 不匹配 | 查 docker 映射与鉴权配置                          |
| 搜索恒为空 / 维度报错                  | embedder 模型或维度不一致  | 全链路统一 embedder (如 1536 维)                 |
| <code>Unknown reranker</code> | SDK 太旧             | <code>pip install --upgrade mem0ai</code> |
| 提取出的事实质量差                     | 提取温度过高 / 模型太弱      | 温度 $\leq 0.2$ ；换强模型；或自定义提取 prompt         |
| 记忆互相串                         | 检索漏了 filters       | 所有 search/get_all 强制带作用域                  |
| 想存原文却存了摘要                     | infer 默认开启         | <code>infer=False</code>                  |

## 9. 动手练习（建议按序完成）

1. **最小验证**：默认配置 add → search，观察提取出的事实与 score
2. **作用域实验**：同一 `user_id` + 两个不同 `agent_id` 读写，验证隔离与共享
3. **冲突实验**：先存“我不吃辣”，再对话“最近开始吃辣了”，观察 DELETE/UPDATE 决策与 `history()`
4. **本地化实验**：Ollama + Chroma 断网跑通，对比提取质量差异
5. **infer 对照**：同一段话分别 `infer=True/False` 入库，比较检索表现

**下一节预告**：单机跑通只是开始，生产怎么接 Agent、控成本、保安全？→ 06-生产篇。

# 06 - 生产篇：集成模式与最佳实践

【本节问题】① 生产 Agent 怎么接入 mem0? ② 怎么和主流框架集成? ③ 和 RAG 怎么共存? ④ 成本、安全、合规怎么管? ⑤ 上线前检查什么?

## 1. 核心循环：Retrieve → Reason → Writeback

mem0 官方推荐的生产架构，**每个请求都跑同一个循环**，接入点只有两个（每步开头的检索、结尾的写回），可增量接入现有 Agent：



## 2. 完整集成示例 (OpenAI + mem0, 可直接改造成你的项目)

```
import os
from openai import OpenAI
from mem0 import MemoryClient

llm_client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])
mem_client = MemoryClient(api_key=os.environ["MEM0_API_KEY"])

AGENT_ID = "support-agent-v1"

def retrieve_memories(user_id: str, query: str, limit: int = 5):
    return mem_client.search(user_id=user_id, agent_id=AGENT_ID,
                             query=query, limit=limit)

def store_memory(user_id: str, text: str, metadata: dict | None = None):
    mem_client.add(user_id=user_id, agent_id=AGENT_ID,
                   text=text, metadata=metadata or {})

def build_prompt(user_message: str, memories: list[dict]) -> str:
    memory_block = ""
    if memories:
        joined = "\n".join(f"- {m['text']}" for m in memories)
        memory_block = f"Known user context:\n{joined}\n\n"
    system = ("You are a helpful support agent. Use the known user context "
              "when relevant, and avoid asking for information already present there.")
    return f"{system}\n\n{memory_block}User: {user_message}\nAssistant:"

def chat(user_id: str, user_message: str) -> str:
    memories = retrieve_memories(user_id, user_message)          # ①
    prompt = build_prompt(user_message, memories)
    resp = llm_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}],
        temperature=0.2,
    )
    answer = resp.choices[0].message.content                    # ②
    store_memory(user_id, f"User said: {user_message}")        # ③
    return answer
```

**工程细节：** - 记忆块要有明确包裹 ( Known user context: ) + system 指示"优先用已知上下文、别重复追问"  
- Writeback 不是无脑全存——挑"值得复用的" (用户偏好、决策、新事实) , 或交给 add 的提取机制把关

## 3. 主流框架集成模式

| 框架                       | 接法                                 | 思路                        |
|--------------------------|------------------------------------|---------------------------|
| OpenAI Agents SDK / 自研循环 | 如上: 工具化两个函数                        | search 作为上下文注入, add 在轮末调用 |
| LangChain / LangGraph    | 把 mem0 包成一个 <b>Tool</b> 或 Graph 节点 | 检索节点放在 LLM 节点前; 写回节点放在结束前 |

| 框架                | 接法           | 思路                                          |
|-------------------|--------------|---------------------------------------------|
| CrewAI            | 官方内置 mem0 集成 | Crew 配置里直接开 memory backend                  |
| mem0 Platform MCP | 走 MCP 协议     | 让任何支持 MCP 的客户端（如 Claude、Cursor 类工具）直接获得记忆能力 |

设计原则（费曼式）：**记忆层是“水电煤”，不是“主角”**。它应该被任何框架以最低成本取用，而不是绑架你的架构。

## 4. 与 RAG 的共存架构

| 维度   | RAG           | 记忆层 (mem0)         |
|------|---------------|--------------------|
| 内容   | 静态知识：文档、手册、政策 | 动态经历：用户事实、会话沉淀     |
| 数据来源 | 语料库，人工/批处理维护  | 用户交互自动沉淀           |
| 更新频率 | 低（内容版本化）      | 高（每次交互都可能写）        |
| 个性化  | 无（对所有人一样）     | 有（按 user/agent 隔离） |
| 典型问题 | "退货政策是什么"     | "我上次买的什么尺码"        |

生产组合拳：



两者互补不替代：RAG 答"世界是什么样"，记忆层答"这个用户经历了什么"。

## 5. 成本优化清单

| 手段    | 说明                                                                         |
|-------|----------------------------------------------------------------------------|
| 控制注入量 | <code>limit</code> 取 3~5 条高相关记忆即可，宁缺毋滥                                     |
| 记忆压缩  | 定期把旧记忆归并/摘要（Platform 的 Memory Compression Engine 自动做；OSS 自建 cron + LLM 压缩） |
| 分层存储  | 一次性信息留在会话层（run_id），别污染长期库                                                  |
| 检索裁剪  | reranker <code>top_k</code> 10~20；无关记忆不进 prompt                            |
| 缓存    | 高频 query 的检索结果短 TTL 缓存                                                     |

## 6. 安全与合规（红线清单）

- ⊙ **绝不存**：密钥、原始凭据、未脱敏 PII（身份证号、卡号）——**记忆层天生为"可检索"而生**，存进去=放进一个必然被查出来的地方
- ✓ 敏感值先**加密或哈希**再入库；展示层脱敏
- ✓ 用户对"被记住"要有知情与清除渠道（对应 GDPR 删除权 → 提供 `delete_all` 能力）
- ✓ 合规背书：mem0 Platform 有 SOC 2 (Type 1)、HIPAA、GDPR；支持 BYOK、审计日志（每次读写可追溯"谁在何时写了什么"）
- ✓ 多租户隔离：显式 `collection_name` / metadata 租户标 + **所有查询强制带 filters**

## 7. 评测：上线前先量，上线后持续量

| 评测对象 | 方法                                                  |
|------|-----------------------------------------------------|
| 检索质量 | 自建 20~50 条 QA：已知某用户记忆，问对应问题，看 top-3 命中率             |
| 提取质量 | 抽 30 段真实对话人工核对：该记的记了吗？不该记的混进来吗？                     |
| 端到端  | 对比"有记忆 vs 无记忆"两版的回答满意度（LLM-as-a-Judge 或人工）          |
| 行业基准 | LoCoMo（长对话记忆）、LongMemEval、BEAM（官方宣称三项均做了 benchmark） |

## 8. 上线检查清单（Copy 即用）

- ☐ 提取 LLM 温度  $\leq 0.2$
- ☐ 生产向量库显式 `collection_name`（租户隔离）
- ☐ 所有 `search/get_all` 强制带作用域 filters（code review 卡点）
- ☐ 敏感数据清洗管线（加密/哈希后再 add）
- ☐ 用户数据删除能力（GDPR）
- ☐ 检索质量基线评测通过（top-3 命中率）
- ☐ 成本监控：记忆注入 token 量、add/search 调用量
- ☐ 记忆治理计划：压缩/剪枝周期、矛盾处理策略
- ☐ 灰度：先对 5% 流量开记忆，对比满意度与成本

## 【本节自测】

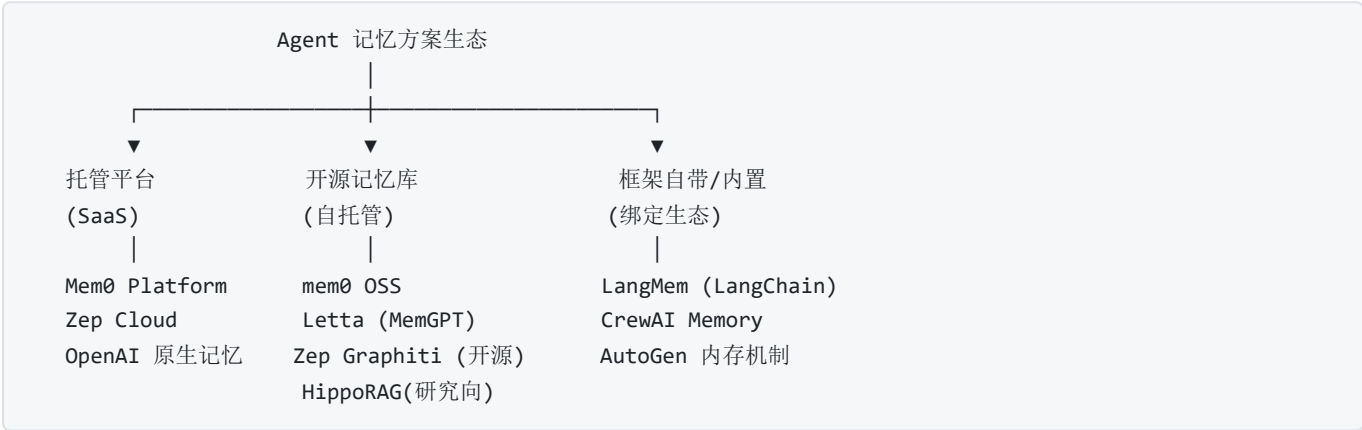
- Retrieve-Reason-Writeback 循环中，两个接入点分别在哪？为什么说可以"增量接入"？
- 记忆注入 prompt 时，prompt 工程上要做哪两件事？
- RAG 和记忆层各管什么？给一个两者必须同时存在的业务例子。
- 为什么"未脱敏 PII 不能存记忆层"？合规上要给用户什么能力？

5. 你的项目上线前，这 9 项检查哪些是必做项？

# 07 - 对比篇：记忆方案横向评测与选型

【本节问题】① Agent 记忆赛道上都有谁、各自什么思路？② 各方案的强项短板？③ 面对具体业务，怎么一步步选出合适的方案？

## 1. 赛道全景图



## 2. 主要方案逐一画像

### 2.1 mem0（本专题主角）

- **思路：**独立记忆层——LLM 提取事实 → 两阶段管线维护 → 向量（+实体加成）混合检索
- **强项：**工程化最好（API 简洁、文档全、OSS+托管双形态）、延迟与 token 开销最低档、LoCoMo 综合表现强、生态接入广（LangChain/CrewAI/MCP...）
- **短板：**带类型的关系推理弱（现役 Graph Memory 是共现图）；OSS 无图记忆
- **适合：**绝大多数需要用户/会话记忆的产品（客服、助理、教育、CRM）

### 2.2 Zep / Graphiti

- **思路：**时序知识图谱——把对话沉淀成带时间边的图，答案要“时间一致”（现在是/曾经是）
- **强项：**开放域与时序问题表现顶尖（LoCoMo 开放域 J=76.60）；关系推理深
- **短板：**token 开销巨大（论文实测 >600k/对话，是原始对话的 20 倍）、记忆可用延迟高（数小时 vs mem0 一分钟内）
- **适合：**知识关系复杂、时间演变重要的场景（企业知识管理、复杂用户关系网络）

### 2.3 Letta（MemGPT 商业化）

- **思路：**LLM 即操作系统——把上下文窗口当内存，记忆分页换入换出，Agent 自己调工具管理记忆
- **强项：**超长会话优雅；记忆管理自主性强；Agent 状态持久化完整
- **短板：**每一步都为“记忆管理”消耗推理 token；学习曲线与运维复杂度高

- **适合**：长周期、自主性强的 Agent（长期研究助理、持续运行的数字员工）

## 2.4 LangMem (LangChain 生态)

- **思路**：LangChain 官方记忆库，与 LangGraph 状态机天然融合
- **强项**：LangChain 用户零摩擦；热/温/冷记忆分层理念清晰
- **短板**：LoCoMo 表现一般 (J=58.10)；强绑定 LangChain 生态
- **适合**：已 All-in LangChain 的团队

## 2.5 HippoRAG (研究向)

- **思路**：模仿海马体记忆索引——PPR（个性化 PageRank）+ 知识图谱做检索
- **强项**：多跳问题学术表现好；开源可复现
- **短板**：偏研究组件，非生产记忆层；无更新/冲突处理等运维能力
- **适合**：论文复现、RAG 增强实验

## 2.6 OpenAI 原生 Memory

- **思路**：平台内置，ChatGPT 自动记住偏好
- **强项**：零成本开箱即用
- **短板**：黑盒、不可控、不可移植、LoCoMo 上 J=52.90 垫底梯队
- **适合**：个人玩具项目；**企业产品基本出局**（数据与行为都不可控）

## 3. 横向对比大表（选型速查）

| 维度       | mem0          | Zep/Graphiti | Letta    | LangMem | OpenAI 记忆 |
|----------|---------------|--------------|----------|---------|-----------|
| 核心架构     | 事实提取+向量(实体加成) | 时序知识图谱       | OS式记忆分页  | 生态内记忆组件 | 黑盒        |
| 记忆表示     | 短事实卡          | 三元组+时间边      | 分层内存     | 分层记忆    | 未知        |
| 检索       | 向量+关键词+实体+时间  | 图遍历+时序       | 分页调取     | 语义      | 未知        |
| token 开销 | ~7k/对话 *      | >600k/对话 X   | 中高（推理消耗） | 中       | 未知        |
| 检索延迟     | p95 0.2s *    | 中            | 中        | 中       | —         |
| 图/关系能力   | 共现图(Platform) | 强 *          | 无        | 无       | 无         |
| 部署形态     | OSS/托管双形态 *   | 开源+云         | 自托管平台    | 库       | 仅SaaS     |
| 数据可控     | OSS 可控 *      | OSS 可控       | 可控       | 可控      | X         |

| 维度          | mem0      | Zep/Graphiti | Letta | LangMem     | OpenAI 记忆 |
|-------------|-----------|--------------|-------|-------------|-----------|
| 生态接入        | 广 *       | 中            | 中     | LangChain 系 | 封闭        |
| LoCoMo 综合 J | 66.9~68.4 | 开放域最强        | —     | 58.1        | 52.9      |

## 4. 选型决策树（照着走）

你的 Agent 需要跨会话记忆吗？  
└ 否 → 不需要记忆层（别过度设计），用会话历史即可  
└ 是 → 数据能出你的服务器/用 SaaS 吗？  
    └ 能 → 预算允许上托管吗？  
        └ 是 → Mem0 Platform（均衡首选）  
            └ 特殊：关系推理极重 → Zep Cloud  
        └ 否 → Mem0 OSS（默认配置起步）  
    └ 否（必须私有化）  
        └ 需要时序/关系图谱推理吗？  
            └ 是 → Zep Graphiti（开源自部署）  
            └ 否 → Mem0 OSS + Qdrant/pgvector  
        └ 已 All-in LangChain? → 先试 LangMem  
        └ 超长会话、Agent 全自主? → 评估 Letta

**默认推荐**（80% 场景）：**Mem0 OSS 起步验证** → **量产后视情况切 Mem0 Platform**。理由：工程化最成熟、开销最低、双形态迁移平滑（API 兼容）。

## 5. 混用策略（进阶）

真实系统常常不止一个方案：

- **mem0（用户记忆） + 传统 RAG（知识库）**：最常见组合，见 06-生产篇 §4
- **mem0（事实记忆） + Letta/自研（工作状态）**：分层各司其职
- **mem0 Platform + 自建向量库（私有语料）**：敏感语料不出域，用户记忆走托管

## 6. 值得跟踪的方向（2026 视角）

- **多模态记忆**：图像/音频交互的沉淀（论文自认的空白）
- **记忆治理**：合规审计、租户级保留策略、遗忘权自动化
- **压缩引擎**：更激进的记忆蒸馏，进一步压 token
- **评测标准化**：LoCoMo / LongMemEval / BEAM 之外更贴近真实业务的基准

## 【本节自测】

1. 用一句话说出五个主要方案各自的核心思路。

2. Zep 在什么题型上最强？代价是什么？
3. MemGPT/Letta 的"操作系统"比喻具体指什么？
4. 为什么企业产品很少用 OpenAI 原生记忆？
5. 给"某银行智能客服（数据不能出域、要求用户偏好长期记忆）"做选型，走一遍决策树。

## 08 - 自测篇：题库与面试题

**用法 (重要)：** 先合上笔记作答，再翻答案。答错的题标记 **✗→✓**，按下方间隔计划复习。**间隔复习计划：** D+1 做 L1 → D+3 做 L2 → D+7 做 L3/L4 → D+21 重做全部错题。

### L1 概念快答 (10 题, 每题 30 秒)

1. mem0 在 Agent 架构中扮演什么角色？它**不负责**什么？
2. mem0 的两个核心 API 是什么？各自在请求周期的什么位置调用？
3. 四层记忆是什么？分别挂什么作用域？
4. `user_id`、`agent_id`、`run_id` 哪个用来实现"自动过期"？
5. `infer=False` 的作用？
6. mem0 的提取/更新决策用四种操作，是哪四种？
7. 默认 LLM、embedding 模型、向量库是什么？
8. Graph Memory 在 OSS 里能用吗？
9. 提取阶段的 LLM 温度建议多少？
10. 哪三类数据绝对不能直接存进记忆层？

### 参考答案

1. 独立记忆层（记忆基础设施）；不负责编排/工具/路由等 Agent 框架职责。
2. `add()`（交互后写入）、`search()`（模型调用前检索）。
3. Conversation（本轮）/ Session（会话, `run_id`）/ User（长期, `user_id`）/ Org（全局共享）。
4. `run_id`。
5. 跳过 LLM 提取，原样存储文本。
6. ADD / UPDATE / DELETE / NOOP。
7. `gpt-5-mini`、`text-embedding-3-small`（1536 维）、本地 Qdrant（`/tmp/qdrant`）。
8. 不能。Graph Memory 是 Platform 功能；OSS 仅实体重合加成。
9.  $\leq 0.2$ （确定性优先）。
10. 密钥/凭据、未脱敏 PII、其他敏感原文（先加密或哈希）。

### L2 原理理解 (6 题, 每题 2 分钟)

1. 完整描述 `add()` 的内部流程（至少 4 步），并说明异步摘要模块在其中的作用。
2. 为什么说 mem0 的自动提取是"增量式"的？这对生产系统意味着什么责任？
3. 检索的四路信号是什么？Platform 和 OSS 在此的差异？

4. 为什么 `score` 不适合当绝对阈值？正确的用法是什么？
5. 论文中 Mem0g 如何构建图、如何双路检索？它为什么时序推理强？
6. 全量上下文在 LoCoMo 上精度最高（72.9%），为什么生产不用？至少三个理由。
7. OSS 检索里，关键词和实体打分能不能往候选池里加新记忆？候选池实际多大？
8. OSS 里 SQLite 承担哪两件事？记忆元数据里 hash 和 lemma 各干什么用？

## 参考答案要点

1. 上下文查找（带最新摘要，防重复/辅助判断新旧）→ LLM 事实提取 → 去重+向量化 → 实体提取/入库；更新阶段按 top-s 相似记忆做 ADD/UPDATE/DELETE/NOOP；摘要后台异步刷新，保证提取时全局上下文最新。
2. 新矛盾事实（"搬到西雅图"）入库时旧事实（"住奥斯汀"）不会被静默改写 → 生产需显式 update/delete 或治理策略，否则新旧矛盾记忆并存会被同时召回。
3. 语义（向量）/ 关键词（BM25）/ 实体（重合或图加成）/ 时间（时间元数据 vs 查询时间意图）；Platform 四路全开且实体由内置图驱动，OSS 仅向量+可选 reranker+实体重合加成。
4. 它是四路信号融合分，不同 query 分布不同，无统一绝对标准；应按相对排序取 top-k，配合 reranker。
5. 两阶段 LLM 管线（实体抽取器→关系生成器）生成带类型三元组存 Neo4j；检索走"实体中心子图探索"+"三元组语义匹配"；结构显式保留时间与关系 → 时序推理强。
6. ① p95 延迟 17.12s vs 1.44s（差 91%）；② token 成本 ~26k/轮 vs ~7k（且每轮重付）；③ 上下文窗口容量硬约束，长对话装不下；④ 换来的精度差距小（~6 点）且 mem0 仍优于绝大多数记忆方案。
7. **不能**——只有 ANN 语义检索负责"出池"，实际召回  $\max(\text{topK} \times 4, 60)$  条（topK=10 → 60 条）；BM25 与实体加分只在池内并行打分；语义(0~1)+关键词(0~1)+实体(0~0.5) 满分 2.5，求和后 ÷ 2.5 归一取 topK。
8. ① 增删改变更日志（对账/回放/溯源），② **最近 10 条消息**（给抽取器消解代词，如 it/he 指代）；hash 只用于**措辞完全相同**的去重；lemma（词形还原版本）供关键词（BM25）检索使用。

## L3 应用设计（4 题，动手/画图）

1. 设计"健身房会员助理"的记忆 schema：哪些信息进 user 层、哪些进 session 层、哪些不存？给出 metadata 设计。
2. 用户先后说"我对海鲜过敏" → 半年后"我现在对海鲜不过敏了"。描述系统的完整行为链（提取→决策→检索→prompt）。
3. 你的 Agent 既要回答产品手册问题（500 页 PDF），又要记得每个客户的沟通历史。画出上下文组装流程图。
4. 把 06-生产篇的 chat() 改造为 LangGraph 节点图：节点怎么分、状态里放什么、写回放哪。

## 参考答案要点

1. user 层：会员等级、过敏史、器械偏好、教练偏好、伤病禁忌；session 层：本次约课流程状态 (run\_id)；不存：一次性寒暄、卡号原文（脱敏/加密后可存引用）。metadata: {"club": "望京店", "member\_tier": "gold"}。
2. "对海鲜过敏"→ADD；半年后"不过敏了"→与新消息相似度最高的旧记忆参与更新决策→LLM 大概率 DELETE 或 UPDATE（改写为"曾对海鲜过敏，现已不再过敏"）；检索"他能吃什么"时时间信号偏向最新事实；prompt 记忆块呈现最新状态；关键：若 LLM 误判未清理，需业务侧提供显式修正入口。
3. 见 06 §4 图：query 同时进 RAG（手册向量检索）与 mem0.search（该客户 user\_id 记忆）→ 两组结果分别标注来源注入 prompt（"产品资料：" / "客户背景："）→ LLM 作答 → 轮末值得记的沟通内容 add 回 mem0。
4. 节点：retrieve\_memory → agent\_reason（含工具循环）→ writeback\_memory；State 含 user\_id、messages、memories(检索结果)、final\_answer；writeback 在 END 前执行，把新事实 add 回去；失败要容错（记忆挂了不能阻塞主对话）。

## L4 面试真题（6 题，含答题框架）

1. "让你给现有客服 Agent 加长期记忆，你怎么设计？" 框架：先问场景（用户回访频率/会话长度）→ 选型（mem0 起步）→ 接入（Retrieve-Reason-Writeback 两接入点，增量改造）→ 治理（作用域/去重/压缩/合规）→ 评测（自建 QA 命中率 + 灰度对比）。
2. "mem0 和 RAG 的区别？什么时候都要用？" 知识 vs 经历；静态 vs 动态；共性 vs 个性化；例子：电商客服（政策 RAG + 该用户订单偏好 mem0）。
3. "记忆层怎么防止用户 A 看到用户 B 的信息？" 作用域强制 filters（code review 卡点）+ 租户级 collection 隔离 + 审计日志 + 权限层校验 user\_id 归属。
4. "mem0 论文里最打动你的数字是什么？怎么验证它适用于你的场景？" 例：p95 1.44s vs 17.12s (91%↓) + token ~7k vs 26k (~73%↓)，精度 -6 点 → 但要自建业务评测集复测，LoCoMo ≠ 业务分布。
5. "用户说'我搬家了'，mem0 会怎么处理？这有什么风险？" 增量 ADD 新事实，旧地址保留；风险：检索同时召回新旧地址 → 依赖更新阶段 DELETE 决策（LLM 判断，非 100% 可靠）→ 强一致场景加版本化+显式修正。
6. "如果让你自研一个记忆层，你会抄 mem0 哪几点、避免哪几点？" 抄：两阶段管线（提取 + ADD/UPDATE/DELETE/NOOP 决策）、作用域模型、四路融合检索、异步摘要；避免/加强：OSS 图能力缺位、冲突检测可靠性、多模态、更激进的压缩与遗忘策略。

## 终极费曼检验（能通过才算真学会）

- [] 给完全没接触过的同事讲 15 分钟"mem0 是什么、怎么工作、什么时候用"（白手画两阶段流水线图）

- [] 凭记忆写出 05-实战篇的 add → search 最小示例并跑通
- [] 不看笔记解释：为什么 LoCoMo 上 mem0 没打过全量上下文，大家还是用它